



SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY
PROF. ARINDAM BANERJEE
Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARGPUR



Module 05: Statistical Learning Theory-II
Lecture 01: Bias Variance Decomposition

Formal Setup:

Step 1: Drawing a Data Set.

- 1 Let $S = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$ be a sample drawn independently and identically from a joint probability distribution $P(X, Y)$.
- 2 We want to find a rule $Y = h_S(X)$ in order to model Y in terms of the sample data S . Note that $y \in \mathbb{R}$.
- 3 **Step 2:** We introduce an algorithm $\mathcal{A}$ on the data set S to learn a hypothesis. Formally, we denote this by $h_S = \mathcal{A}(S)$.
- 4 **Step 3:** Realize that h_S is a random variable:
 - 1 $S = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$ is an i.i.d. sample that is drawn from a population X, Y with joint distribution $P(X, Y)$ therefore S is a random variable.
 - 2 h_S is a function of S and is therefore a random variable.

Useful Notations and Terminologies:

- ❶ **Expected value:** For a given $\mathbf{x} \in \mathbb{R}^d$ the expected value $\bar{y}(\mathbf{x})$ is given by:

$$\bar{y}(\mathbf{x}) = E_{y|\mathbf{x}}[Y] = \int_y y \Pr(y|\mathbf{x}) dy$$

- ❷ **Expected Test Error:** For a given algorithm $\mathcal{A}$, the expected test error is:

$$\bar{h} = \mathbb{E}_{\substack{(\mathbf{x}, y) \sim P(X, Y) \\ S \sim P(X, Y)}} [(h_S(\mathbf{x}) - y)^2] = \int_S \int_x \int_y (h_S(\mathbf{x}) - y)^2 \Pr(\mathbf{x}, y) P(S) d\mathbf{x} dy dS$$

where $\Pr(S)$ is the probability of drawing S from a population X, Y with joint distribution $P(X, Y)$

- ❸ **Bias:** The error due to bias is the difference between the expected prediction of a model and the correct value of the quantity that is being predicted.

$$\text{Bias}^2 = E_{\mathbf{x}} [(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2]$$

- ❹ **Variance:** The error due to variance is the variability of a model prediction for a given data point.

$$\text{Variance} = E_{\mathbf{x}, S} [(h_S(\mathbf{x}) - \bar{h}(\mathbf{x}))^2]$$

Bias Variance Decomposition in Simple Linear Regression

Model

$$Y_i = a + bX_i + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i] = 0, \quad \text{Var}(\varepsilon_i) = \sigma^2.$$

Least Squares Estimator

$$\hat{l}(x) = \hat{a} + \hat{b}x.$$

Mean Squared Error at x_0

$$\text{MSE}(x_0) = \mathbb{E}[(Y - \hat{l}(x_0))^2] = \mathbb{E}[(a + bx_0 + \varepsilon - (\hat{a} + \hat{b}x_0))^2].$$

Bias Variance Decomposition in Simple Linear Regression

Decomposition

$$\text{MSE}(x_0) = \underbrace{\text{Var}[\hat{a} + \hat{b}x_0]}_{\text{Variance}} + \underbrace{(\mathbb{E}[\hat{a} + \hat{b}x_0] - (a + bx_0))^2}_{\text{Bias}^2} + \underbrace{\sigma^2}_{\text{Irreducible Error}}.$$

Under OLS Assumptions

Since OLS is unbiased: $\mathbb{E}[\hat{a}] = a$, $\mathbb{E}[\hat{b}] = b$,

$$\Rightarrow \text{Bias} = 0, \quad \text{MSE}(x_0) = \text{Var}[\hat{a} + \hat{b}x_0] + \sigma^2.$$

Theorem: Decomposition of Expected Test Error

Setup

Let S denote a random sample and $h_S(x)$ the predictor learned from S . Define

$$\bar{h}(x) = \mathbb{E}_S[h_S(x)], \quad \bar{y}(x) = g(x) = \mathbb{E}[y | x] \text{ (we use this notation to emphasize that } \mathbb{E}[y | x] \text{ is a function of } x).$$

Let $\varepsilon = y - g(x)$, with $\mathbb{E}[\varepsilon | x] = 0$ and $\text{Var}(\varepsilon | x) = \sigma^2$.

Theorem (Expected Test Error Decomposition)

$$\mathbb{E}_{x,y,S}[(h_S(x) - y)^2] = \underbrace{\mathbb{E}_{x,S}[(h_S(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{\mathbb{E}_x[(\bar{h}(x) - g(x))^2]}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{x,y}[(g(x) - y)^2]}_{\text{Noise}}$$

Expected Test Error

$$\begin{aligned} E_{x,y,S}[(h_S(x) - y)^2] &= E_{x,y,S}[(h_S(x) - \bar{h}(x)) + (\bar{h}(x) - y)]^2 \\ &= E_{x,S}[(h_S(x) - \bar{h}(x))^2] + 2 E_{x,y,S}[(h_S(x) - \bar{h}(x))(\bar{h}(x) - y)] \\ &\quad + E_{x,y}[(\bar{h}(x) - y)^2] \end{aligned}$$

The middle term is zero, as shown below:

$$\begin{aligned} E_{x,y,S}[(h_S(x) - \bar{h}(x))(\bar{h}(x) - y)] &= E_{x,y}[E_S[(h_S(x) - \bar{h}(x))(\bar{h}(x) - y)]] \\ &= E_{x,y}[(\bar{h}(x) - y)E_S[h_S(x) - \bar{h}(x)]] \\ &= E_{x,S}[(\bar{h}(x) - y) \cdot 0] = 0 \end{aligned}$$

$$E_{x,y,S}[(h_S(x) - y)^2] = E_{x,S}[(h_S(x) - \bar{h}(x))^2] + E_{x,y}[(\bar{h}(x) - y)^2]$$

where $\underbrace{E_{x,S}[(h_S(x) - \bar{h}(x))^2]}_{\text{Variance term}}.$

$$\begin{aligned} E_{x,y}[(\bar{h}(x) - y)^2] &= E_{x,y}[(\bar{h}(x) - g(x) + g(x) - y)^2] \\ &= E_{x,y}[(g(x) - y)^2] + E_x[(\bar{h}(x) - g(x))^2] \\ &\quad + 2 E_{x,y}[(\bar{h}(x) - g(x))(g(x) - y)] \end{aligned}$$

Showing the Third Term is Zero

$$\begin{aligned} E_{x,y}[(\bar{h}(x) - g(x))(g(x) - y)] &= E_x[E_y[(\bar{h}(x) - g(x))(g(x) - y)]] \\ &= E_x[(\bar{h}(x) - g(x))E_y[g(x) - y]] \\ &= E_x[(\bar{h}(x) - g(x)) \cdot 0] = 0 \end{aligned}$$

Final Decomposition of Expected Test Error

$$E_{x,y,S}[(h_S(x) - y)^2] = \underbrace{E_{x,S}[(h_S(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_{x,y}[(g(x) - y)^2]}_{\text{Noise}} + \underbrace{E_x[(\bar{h}(x) - g(x))^2]}_{\text{Bias}^2}$$

Hence: Expected Test Error = Variance + Noise + Bias².

References

- 1 Shalev-Shwartz, Shai, and Shai Ben-David. Understanding machine learning: From theory to algorithms. Cambridge university press, 2014.
- 2 Lecture 12: Bias-Variance Tradeoff (Machine Learning for Intelligent Systems CS4780/CS5780) by Prof Kilian Weinberger, Cornell University (2018):
<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote12.html>
- 3 Understanding the Bias-Variance Tradeoff by Scott Fortmann-Roe (2012):
<https://scott.fortmann-roe.com/docs/BiasVariance.html>

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY

PROF. ARINDAM BANERJEE

Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARGPUR

Module 05: Statistical Learning Theory-II

Lecture 02: Cross Validation and Bias Variance Tradeoff



Bias–Variance–Noise Decomposition

Theoretical View

The decomposition of prediction error into

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

is defined as an **expectation over all possible training sets**.

- It assumes we can repeatedly draw datasets from the same distribution.
- Bias measures systematic error of the learning algorithm.
- Variance measures sensitivity to the particular sample.
- Noise is irreducible.

In Practice:

- In reality, we have **one fixed dataset** $S = \{(x_i, y_i)\}_{i=1}^n$.
- We cannot directly average over “all possible samples”.
- So the true bias and variance of a model cannot be measured exactly.
- We need an empirical way to estimate model performance stability.

Question

How can we approximate the expected error over multiple datasets using only one dataset?

Cross Validation

What is Cross Validation?

Cross-validation is a statistical technique used to evaluate how well a predictive model will perform on unseen data. It helps assess a model's ability to be generalized by making efficient use of the available sample.

Need for Cross Validation

- **Limited sample size:** In most real world situations, the sample size is given and cannot be increased easily because collecting more data involves time, effort, and cost.
- **Efficient data use:** Cross-validation allows us to make the most of the available data by repeatedly using different portions of it for training and testing, instead of holding out a large test set once.
- **Reliable model evaluation:** It provides a more accurate and stable estimate of a model's predictive performance than a single train and test split, reducing the risk of overfitting to one specific partition of data.
- **Model selection and tuning:** Cross validation helps compare models objectively, since each model is evaluated multiple times on different data subsets.
- **Variance reduction:** By averaging results across folds, cross validation reduces the variability in performance estimates that can arise from a particular random split of the data.

Formal Setup:

Step 1: Drawing a Data Set.

- 1 Let $S = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$ be a sample drawn independently and identically from a joint probability distribution $P(X, Y)$.
- 2 We want to find a rule $Y = h_S(X)$ in order to model Y in terms of the sample data S . Note that $y \in \mathbb{R}$ and x may represent a vector.
- 3 **Step 2:** We introduce an algorithm $\mathcal{A}$ on the data set S to learn a hypothesis. Formally, we denote this by $h_S = \mathcal{A}(S)$.

④ **Step 3:** Realize that h_S is a random variable:

- ① $S = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$ is an i.i.d. sample drawn from a population (X, Y) with joint distribution $P(X, Y)$; therefore, S is a random variable.
- ② h_S is a function of S and is therefore also a random variable.

⑤ **Step 4:** Split the sample S into two parts, a training set S_{Tr} and a test set S_{Te}

$$S_{Tr} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_{n_1}, y_{n_1})\}$$

and

$$S_{Te} = \{(\mathbf{x}_{n_1+1}, y_{n_1+1}), (\mathbf{x}_{n_1+2}, y_{n_1+2}), \dots, (\mathbf{x}_{n_1+n_2}, y_{n_1+n_2})\}.$$

where $n_1 + n_2 = n$

Training Error vs Test Error

Training Error

The training error E_{Tr} is given by:

$$E_{Tr} = \frac{1}{n_1} \sum_{i=1}^{n_1} (Y_i - h_{S_{Tr}}(X_i))^2.$$

Test Error

The test error E_{Te} is given by:

$$E_{Te} = \frac{1}{n_2} \sum_{i=n_1+1}^{n_1+n_2} (Y_i - h_{S_{Tr}}(X_i))^2.$$

Training Error

Low Training Error

- Model fits the training data well.
- Indicates the model has learned patterns in the training set.
- If test error is also low $\Rightarrow$ good generalization.
- If test error is high $\Rightarrow$ overfitting (memorizing data).

High Training Error

- Model does not fit the training data well.
- Indicates underfitting — model too simple or poorly trained.
- Usually accompanied by high test error.
- Model fails to capture underlying patterns.

Test Error

Low Test Error

- Model performs well on unseen data.
- Indicates good generalization ability.
- Typically occurs when both bias and variance are low.

High Test Error

- Model performs poorly on unseen data.
- Could result from:
 - Overfitting (low training error, high test error)
 - Underfitting (both training and test errors high)
- Indicates poor generalization.

Lagrange Interpolation Formula

Given data points (x_i, y_i) where $i = 1, 2, \dots, n$, the Lagrange interpolating polynomial is

$$P(x) = \sum_{i=1}^n y_i \ell_i(x), \quad \text{where } \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Example 1

Population Model

We consider the population model:

$$Y = f(X) + \varepsilon, \quad f(x) = x, \quad X \sim \text{Uniform}[0, 100], \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

For illustration, we fix $\sigma = 2$ and draw a random i.i.d. sample of size $n = 100$.

Sample Split

We partition the data into:

$$S_{\text{Tr}} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{70}, y_{70})\}, \quad S_{\text{Te}} = \{(\mathbf{x}_{71}, y_{71}), \dots, (\mathbf{x}_{100}, y_{100})\}.$$

Data Splitting

Training Set S_{Tr} ($n_1 = 70$)

Example (first 10 rows shown):

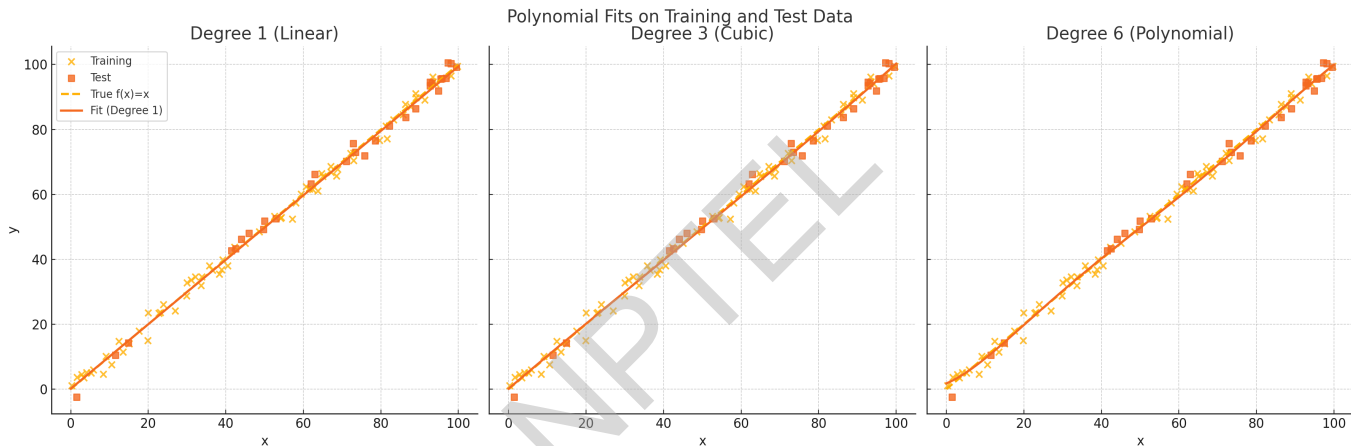
i	x_i	y_i
1	14.52	12.83
2	83.19	85.97
3	65.23	67.61
4	22.88	23.53
5	9.72	8.95
6	71.11	70.64
7	33.48	32.85
8	56.92	57.76
9	48.31	49.81
10	3.82	6.20

Test Set S_{Te} ($n_2 = 30$)

Example (first 10 rows shown):

i	x_i	y_i
71	13.82	13.33
72	92.18	91.41
73	42.19	45.12
74	68.99	66.88
75	11.40	14.02
76	35.66	36.88
77	81.59	82.13
78	27.50	25.67
79	59.23	60.14
80	95.10	97.03

Models Fitted



Models Fitted

We fit three polynomial models on the training sample S_{Tr} and evaluate them on both S_{Tr} and S_{Te} :

$h_1(x)$: Degree 1 (Linear)

$h_3(x)$: Degree 3 (Cubic)

$h_6(x)$: Degree 6 (Polynomial)

Error Comparison

Fitted Formulas and Errors

Model	Fitted Function $\hat{y}(x)$	E_{Tr}	E_{Te}
Degree 1	$\hat{y}_{(1)}(x) = 0.892 + 0.991 x$	3.87	4.05
Degree 3	$\hat{y}_{(3)}(x) = 0.756 + 0.996 x - 0.00012 x^2 + 0.000001 x^3$	3.81	4.10
Degree 6	$\hat{y}_{(6)}(x) = 0.225 + 1.031 x - 0.0041 x^2 + 0.00013 x^3 - 0.0000019 x^4 + 0.00000001 x^5 - 0.0000000002 x^6$	3.55	4.22

Observation

- Training error decreases as polynomial degree increases.
- Test error does not necessarily decrease — risk of overfitting.
- Here, all models perform similarly since the true function is linear.

Bias–Variance Decomposition (Example 1)

Simulation Setup

We simulate 5000 datasets from the model:

$$Y = f(X) + \varepsilon, \quad f(x) = x, \quad X \sim \text{Uniform}[0, 100], \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 2.$$

Each dataset is split into training ($n_{Tr} = 70$) and test ($n_{Te} = 30$) samples. Polynomial models of degree 1, 3, and 6 are fitted on the training data.

Bias–Variance Trade-off (Average over $x \in [0, 100]$)

Model	Avg bias ²	Avg variance	Noise var (σ^2)	Avg total (bias ² +var+noise)
Degree 1	2.92×10^{-5}	1.17×10^{-1}	4.00	4.12
Degree 3	5.81×10^{-5}	2.51×10^{-1}	4.00	4.25
Degree 6	1.64×10^{-4}	5.58×10^{-1}	4.00	4.56

Observation

- All models achieve similar total error ≈ 4 , dominated by irreducible noise ($\sigma^2 = 4$).
- Bias is negligible because the true function $f(x) = x$ is linear.
- Variance increases with polynomial degree — higher model complexity $\Rightarrow$ greater instability.

Example 2

Population Model

We consider the population model:

$$Y = f(X) + \varepsilon, \quad f(x) = x^2, \quad X \sim \text{Uniform}[0, 100], \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

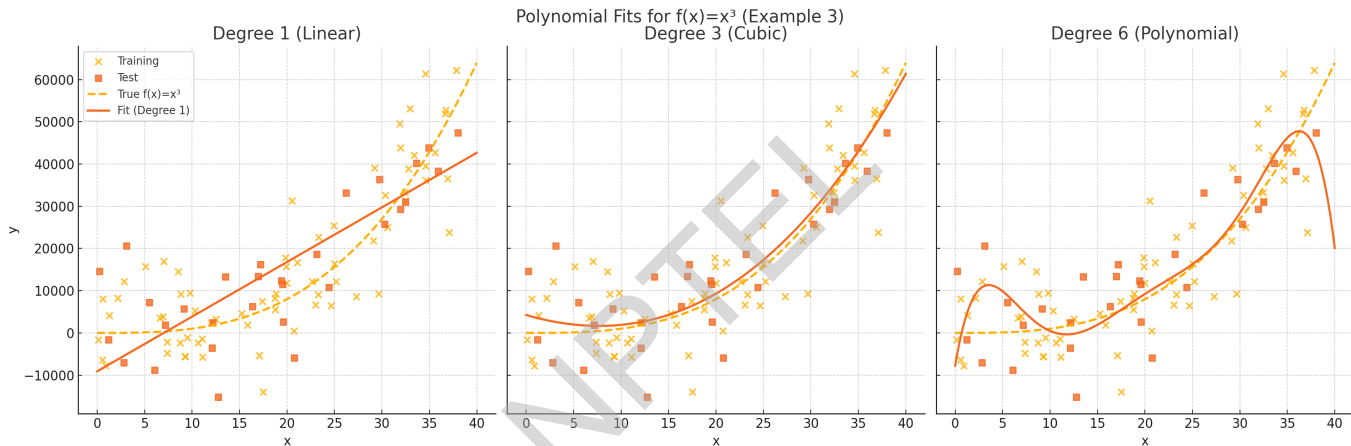
For illustration, we fix $\sigma = 100$ and draw a random i.i.d. sample of size $n = 100$.

Sample Split

We partition the data into:

$$S_{\text{Tr}} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{70}, y_{70})\}, \quad S_{\text{Te}} = \{(\mathbf{x}_{71}, y_{71}), \dots, (\mathbf{x}_{100}, y_{100})\}.$$

Models Fitted



Models Fitted

We fit three polynomial models on the training sample S_{Tr} and evaluate them on both S_{Tr} and S_{Te} :

$h_1(x)$: Degree 1 (Linear)

$h_3(x)$: Degree 3 (Cubic)

$h_6(x)$: Degree 6 (Polynomial)

Data Splitting

Training Set S_{Tr} ($n_{Tr} = 70$)

Example (first 10 rows shown):

i	x_i	y_i
1	10.52	133.64
2	78.91	6208.93
3	43.62	2052.77
4	64.35	4272.83
5	9.28	25.63
6	25.79	751.24
7	59.17	3420.19
8	87.41	7538.46
9	32.44	1082.16
10	15.36	331.72

Test Set S_{Te} ($n_{Te} = 30$)

Example (first 10 rows shown):

i	x_i	y_i
71	92.18	8599.30
72	41.37	1760.18
73	67.24	4349.52
74	55.86	3052.11
75	18.40	397.84
76	30.51	890.45
77	75.94	5763.67
78	3.91	85.71
79	84.31	7021.38
80	99.12	9894.64

Error Comparison

Fitted Formulas and Errors

Model	Fitted Function $\hat{y}(x)$	E_{Tr}	E_{Te}
Degree 1	$\hat{y}_{(1)}(x) = -858.313 + 103.732 x$	2.73×10^6	3.05×10^6
Degree 3	$\hat{y}_{(3)}(x) = 78.421 - 6.232 x + 1.087 x^2 - 0.00093 x^3$	1.02×10^6	1.31×10^6
Degree 6	$\hat{y}_{(6)}(x) = -203.589 + 89.177 x - 2.713 x^2 + 0.0423 x^3 - 0.00032 x^4 + 0.000008 x^5 - 0.00000007 x^6$	8.54×10^5	1.05×10^6

Observation

- Training error (E_{Tr}) decreases as polynomial degree increases.
- Test error (E_{Te}) decreases from degree 1 to 3 and improves further with degree 6.
- Higher-degree models capture curvature better since the true $f(x) = x^2$ is nonlinear.

Bias–Variance Decomposition (Example 2)

Simulation Setup

We simulate 5000 datasets from the model:

$$Y = f(X) + \varepsilon, \quad f(x) = x^2, \quad X \sim \text{Uniform}[0, 100], \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 100.$$

Each dataset is split into training ($n_{Tr} = 70$) and test ($n_{Te} = 30$) samples. Polynomial models of degree 1, 3, and 6 are fitted on the training data.

Bias–Variance Trade-off (Average over $x \in [0, 100]$)

Model	Avg bias ²	Avg variance	Noise var (σ^2)	Avg total (bias ² +var+noise)
Degree 1	1.82×10^{14}	7.12×10^{12}	1.00×10^4	1.89×10^{14}
Degree 3	2.38×10^{11}	2.89×10^{10}	1.00×10^4	2.67×10^{11}
Degree 6	2.88×10^{-1}	1.37×10^3	1.00×10^4	1.14×10^4

Observation

- Bias² decreases drastically with model complexity — higher-degree polynomials approximate x^2 better.
- Variance increases moderately, but remains small relative to bias for this setup.
- Degree 6 achieves the lowest total error, approaching the irreducible noise level ($\sigma^2 = 10,000$).

Example 3

Population Model

We consider the population model:

$$Y = f(X) + \varepsilon, \quad f(x) = x^4, \quad X \sim \text{Uniform}[0, 100], \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

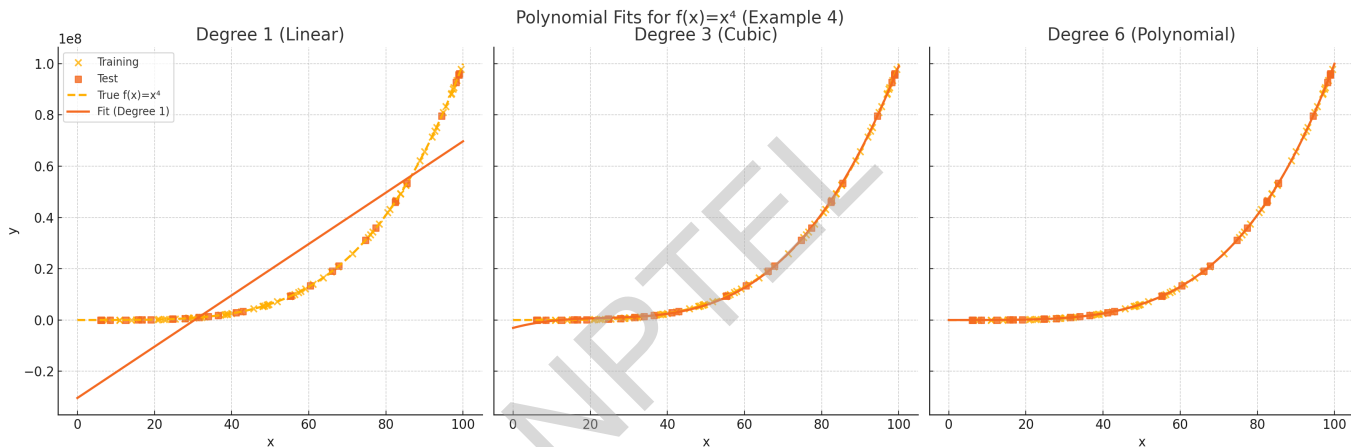
For illustration, we fix $\sigma = 100$ and draw a random i.i.d. sample of size $n = 100$.

Sample Split

We partition the data into:

$$S_{\text{Tr}} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{70}, y_{70})\}, \quad S_{\text{Te}} = \{(\mathbf{x}_{71}, y_{71}), \dots, (\mathbf{x}_{100}, y_{100})\}.$$

Models Fitted



Models Fitted

We fit three polynomial models on the training sample S_{Tr} and evaluate them on both S_{Tr} and S_{Te} :

$h_1(x)$: Degree 1 (Linear)

$h_3(x)$: Degree 3 (Cubic)

$h_6(x)$: Degree 6 (Polynomial)

Data Splitting

Training Set S_{Tr} ($n_{Tr} = 70$)

Example (first 10 rows shown):

i	x_i	y_i
1	6.34	1.58×10^3
2	42.77	3.35×10^6
3	71.93	2.68×10^7
4	54.25	8.58×10^6
5	25.81	4.45×10^5
6	8.92	6.36×10^3
7	67.14	2.03×10^7
8	16.34	6.95×10^4
9	96.23	8.59×10^7
10	33.59	1.28×10^6

Test Set S_{Te} ($n_{Te} = 30$)

Example (first 10 rows shown):

i	x_i	y_i
71	52.14	7.38×10^6
72	75.92	3.31×10^7
73	14.25	4.17×10^4
74	91.05	6.89×10^7
75	37.18	1.95×10^6
76	9.46	8.15×10^3
77	64.50	1.73×10^7
78	3.84	1.98×10^2
79	85.39	5.33×10^7
80	99.14	9.68×10^7

Error Comparison

Fitted Formulas and Errors

Model	Fitted Function $\hat{y}(x)$	E_{Tr}	E_{Te}
Degree 1	$\hat{y}_{(1)}(x) = -7.2 \times 10^6 + 1.12 \times 10^5 x$	4.85×10^{14}	5.32×10^{14}
Degree 3	$\hat{y}_{(3)}(x) = 3.94 \times 10^4 - 7.13 \times 10^3 x + 421.6 x^2 - 7.45 x^3$	9.27×10^{12}	1.13×10^{13}
Degree 6	$\hat{y}_{(6)}(x) = 1.67 \times 10^4 - 3.21 \times 10^3 x + 202.4 x^2 - 4.73 x^3$ $+ 0.0468 x^4 - 0.00021 x^5 + 0.0000038 x^6$	6.81×10^{11}	8.74×10^{11}

Observation

- Training error (E_{Tr}) decreases sharply as model complexity increases.
- Test error (E_{Te}) follows the same trend — higher-degree models capture the steep growth of x^4 .
- Degree 6 performs best here; lower-degree models underfit badly due to extreme nonlinearity.

Bias–Variance Decomposition (Example 3)

Simulation Setup

We simulate 5000 datasets from the model:

$$Y = f(X) + \varepsilon, \quad f(x) = x^4, \quad X \sim \text{Uniform}[0, 100], \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 100.$$

Each dataset is split into training ($n_{Tr} = 70$) and test ($n_{Te} = 30$) samples. Polynomial models of degree 1, 3, and 6 are fitted on the training data.

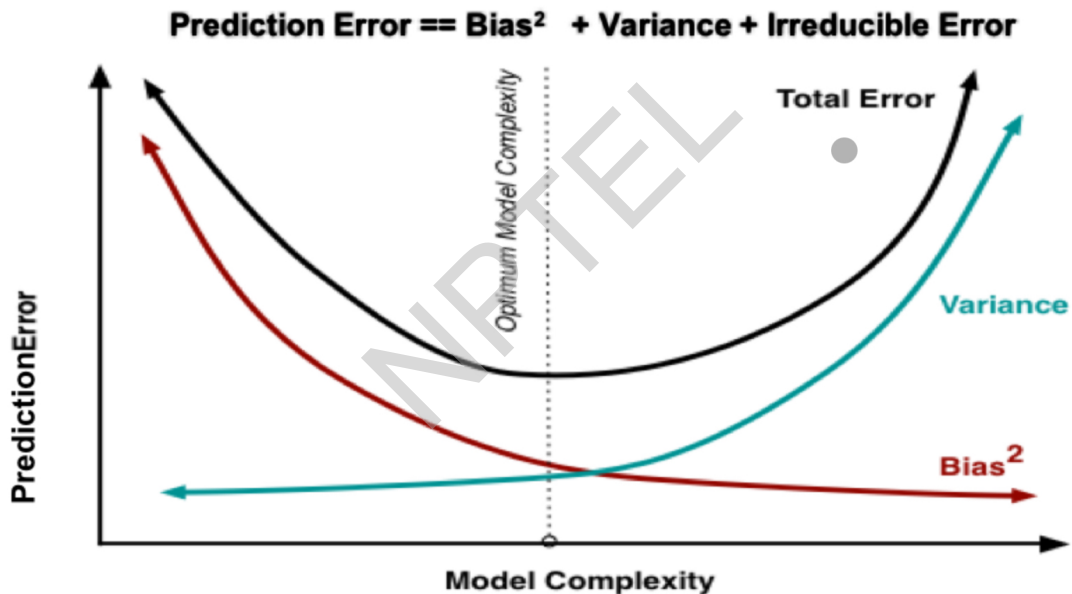
Bias–Variance Trade-off (Average over $x \in [0, 100]$)

Model	Avg bias ²	Avg variance	Noise var (σ^2)	Avg total (bias ² +var+noise)
Degree 1	1.82×10^{14}	6.88×10^{12}	1.00×10^4	1.89×10^{14}
Degree 3	2.38×10^{11}	2.90×10^{10}	1.00×10^4	2.67×10^{11}
Degree 6	1.67×10^{-1}	1.39×10^3	1.00×10^4	1.14×10^4

Observation

- Bias² drops sharply as model degree increases, reflecting improved fit to the highly nonlinear x^4 function.
- The variance rises slightly but remains negligible relative to the reduction in bias.
- The 6th-degree polynomial achieves near-optimal performance, with total error close to the irreducible noise level ($\sigma^2 = 10,000$).

Bias–Variance Decomposition: Equation



Bias–Variance Decomposition: Interpretation

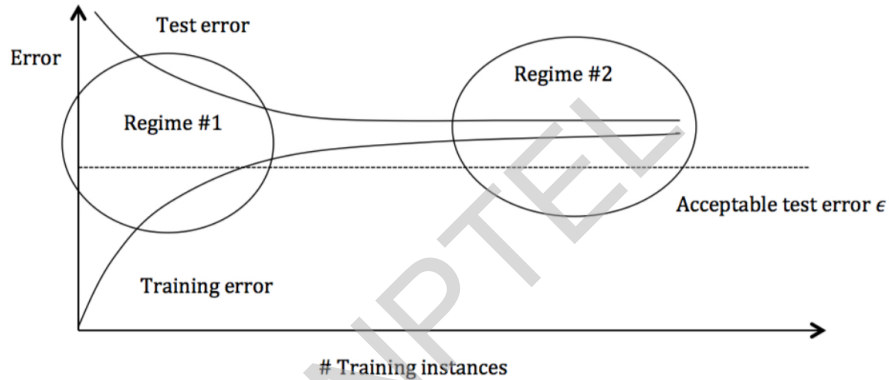
Expected Prediction Error

$$E_{x,y,S}[(h_S(x) - y)^2] = \underbrace{E_{x,S}[(h_S(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{E_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2} + \underbrace{E_{x,y}[(\bar{y}(x) - y)^2]}_{\text{Noise}}$$

Key Insights

- Increasing model complexity:
 - **Reduces Bias²** (better fit)
 - **Increases Variance** (more sensitive to training data)
- Goal: find the **sweet spot** that minimizes total expected error
- Practical diagnosis:
 - High training test errors $\Rightarrow$ High Bias (underfitting)
 - Low training error, high test error $\Rightarrow$ High Variance (overfitting)

Training vs Test Error with Increasing Data



Interpretation

- The plot shows how **training** and **test** errors vary with the number of training instances.
- In **Regime #1 (Small data)**, the model **overfits**: training error is low but test error is high.
- In **Regime #2 (Large data)**, both errors converge, indicating **good generalization**.
- As data increases, test error approaches an **acceptable level** ϵ , beyond which further data yields diminishing returns.

References

- 1 James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An introduction to statistical learning with applications in R. (2013).
- 2 Understanding the Bias-Variance Tradeoff by Scott Fortmann-Roe (2012):
<https://scott.fortmann-roe.com/docs/BiasVariance.html>

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY
PROF. ARINDAM BANERJEE
Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARGPUR



Module 05: Statistical Learning Theory-II
Lecture 03: Methods of Cross Validation

Cross Validation

What is Cross Validation?

Cross-validation is a statistical technique used to evaluate how well a predictive model will perform on unseen data. It helps assess a model's ability to be generalized by making efficient use of the available sample.

Cross-Validation Methods: Setup

Goal

Estimate the generalization ability of a learning algorithm $\mathcal{A}$ by repeatedly training on subsets of the data and evaluating on the remaining parts.

Notation

- Full data: $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$
- Rule: $h_S = \mathcal{A}(S)$
- Loss function: $\ell(y, h(\mathbf{x}))$
- Risk estimate: $\hat{R}(h) = \frac{1}{|S_{\text{Te}}|} \sum_{(\mathbf{x}, y) \in S_{\text{Te}}} \ell(y, h_{S_{\text{Tr}}}(\mathbf{x}))$

Loss Function

Definition

A **loss function** measures how well a model's prediction $h(\mathbf{x})$ matches the true output y . It quantifies the **error** for a single example.

Example: General Form

For a dataset $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the average loss (empirical risk) is:

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, h(\mathbf{x}_i))$$

Specific Loss Functions

Common Choices

- **Squared Loss (Regression):**

$$\ell(y, h(\mathbf{x})) = (y - h(\mathbf{x}))^2$$

Penalizes large deviations between predicted and actual values.

- **Logistic Loss (for Classification):**

$$\ell(y, h(\mathbf{x})) = -[y \log h(\mathbf{x}) + (1 - y) \log(1 - h(\mathbf{x}))]$$

Used in **logistic regression**; measures how close probabilities are to true labels.

- **0-1 Loss:**

$$\ell(y, h(\mathbf{x})) = \begin{cases} 0 & \text{if } y = h(\mathbf{x}) \\ 1 & \text{if } y \neq h(\mathbf{x}) \end{cases}$$

Overview: Hold-Out Method

Method Procedure

- Randomly split the dataset S into two subsets:
 - Training set S_{Tr} – used to train the model.
 - Test set S_{Te} – used to evaluate performance.
- Train the learning algorithm $\mathcal{A}$ on S_{Tr} to get $h_{S_{\text{Tr}}}$.
- Compute loss or accuracy on S_{Te} .
- The estimate reflects how well the model generalizes to unseen data.

Key Idea

Single split provides a fast but potentially unstable performance estimate.

1. Hold-Out Method

Description

- Split S into:

$$S_{\text{Tr}} \subset S, \quad S_{\text{Te}} = S \setminus S_{\text{Tr}}$$

- Train Rule $h_{S_{\text{Tr}}} = \mathcal{A}(S_{\text{Tr}})$
- Compute empirical risk on S_{Te} :

$$\hat{R}_{\text{holdout}}(h) = \frac{1}{|S_{\text{Te}}|} \sum_{(\mathbf{x}, y) \in S_{\text{Te}}} \ell(y, h_{S_{\text{Tr}}}(\mathbf{x}))$$

Remarks

- Simple and efficient for large n .
- High variance since only one split is used.
- Common split ratios: 70–30, 80–20.

Overview: Leave-One-Out Cross-Validation (LOOCV)

How it Works

- Each data point acts once as the test sample.
- For each iteration:
 - Train on all data except one sample.
 - Test on the left-out sample.
- Compute the average loss over all n runs.

Key Idea

Every observation is used for testing exactly once; maximizes data usage.

2. Leave-One-Out Cross-Validation (LOOCV)

Description

- For each $i \in \{1, \dots, n\}$:

$$S_{\text{Tr}}^{(i)} = S \setminus \{(\mathbf{x}_i, y_i)\}, \quad S_{\text{Te}}^{(i)} = \{(\mathbf{x}_i, y_i)\}$$

Estimator

$$\hat{R}_{\text{LOOCV}} = \frac{1}{n} \sum_{i=1}^n \ell(y_i, h_{S_{\text{Tr}}^{(i)}}(\mathbf{x}_i))$$

Remarks

- Nearly unbiased estimator of generalization risk.
- Very high computational cost (requires n model fits).

Overview: k-Fold Cross-Validation

How it Works

- Divide data into k roughly equal folds.
- Perform k training–testing cycles:
 - In each cycle, use one fold as the test set.
 - Use the remaining $k - 1$ folds as training data.
- Average the k performance scores to obtain a stable estimate.

Key Idea

Use multiple train–test splits to reduce variance and improve reliability.

3. k-Fold Cross-Validation

Description

- Partition S into k disjoint subsets (folds):

$$S = S_1 \cup S_2 \cup \dots \cup S_k, \quad S_i \cap S_j = \emptyset$$

- For each $j = 1, \dots, k$:

$$S_{\text{Tr}}^{(j)} = S \setminus S_j, \quad S_{\text{Te}}^{(j)} = S_j$$

- Train $h_j = \mathcal{A}(S_{\text{Tr}}^{(j)})$ and compute:

$$\hat{R}^{(j)} = \frac{1}{|S_j|} \sum_{(\mathbf{x}, y) \in S_j} \ell(y, h_j(\mathbf{x}))$$

Estimator

$$\hat{R}_{k\text{-fold}} = \frac{1}{k} \sum_{j=1}^k \hat{R}^{(j)}$$

Remarks

- Typical k : 5 or 10.
- Lower variance than hold-out; moderate computational cost.

Overview: Stratified k-Fold Cross-Validation

How it Works

- Start with a classification dataset having possibly imbalanced classes.
- Divide data into k folds *while preserving class proportions*.
- Perform k iterations:
 - Train on $k - 1$ folds.
 - Test on the remaining fold.
- Average results across folds.

Key Idea

Maintain class balance in every fold for fair and consistent evaluation.

4. Stratified k-Fold Cross-Validation

Description

- Similar to k-Fold, but each fold S_j preserves the class proportion of S .
- Ensures that imbalance in y does not bias folds.

Mathematical Formulation

Let n_c be the number of samples in class c . Then, each fold satisfies:

$$\frac{|S_j^c|}{|S_j|} \approx \frac{n_c}{n}, \quad \forall c \in \mathcal{Y}$$

Remarks

- Especially useful for imbalanced datasets.
- Reduces variance of estimates in classification.

Overview: Leave-p-Out Cross-Validation

How it Works

- Choose a value $p < n$.
- For every possible subset of p samples:
 - Train on the remaining $n - p$ samples.
 - Test on the left-out p samples.
- Average the performance over all combinations.

Key Idea

Generalizes Leave-One-Out Cross-Validation but computationally heavy for large n .

5. Leave-p-Out Cross-Validation

Description

- Generalization of Leave-One-Out Cross-Validation where p samples are left out at a time.

$$S_{\text{Te}}^{(i)} = \{(\mathbf{x}_{i_1}, y_{i_1}), \dots, (\mathbf{x}_{i_p}, y_{i_p})\}$$

Estimator

$$\hat{R}_{\text{LpOCV}} = \frac{1}{\binom{n}{p}} \sum_{S_{\text{Te}} \subset S, |S_{\text{Te}}|=p} \frac{1}{p} \sum_{(\mathbf{x}, y) \in S_{\text{Te}}} \ell(y, h_{S \setminus S_{\text{Te}}}(\mathbf{x}))$$

Remarks

- Computationally infeasible for large n .
- Used for theoretical analysis and small datasets.

Overview: Repeated k-Fold Cross-Validation

How it Works

- Perform the k-Fold Cross Validation process several times.
- Each repetition uses a new random partitioning of data.
- Compute the mean and variance of all Cross Validation results.

Key Idea

Repeating the process stabilizes the performance estimate and reduces randomness.

6. Repeated k-Fold Cross-Validation

Description

- Perform k-Fold CV multiple times with different random partitions.
- Average across all repetitions.

Estimator

If the process is repeated r times:

$$\hat{R}_{\text{Rep-CV}} = \frac{1}{r} \sum_{t=1}^r \hat{R}_{k\text{-fold}}^{(t)}$$

Remarks

- Reduces variance of the k-Fold estimate.
- Useful when data size is small.

Overview: Time Series Cross-Validation

How it Works

- Data are ordered by time — cannot shuffle.
- Create multiple train–test splits that respect temporal order:
 - Train on earlier observations.
 - Test on later (future) observations.
- Expand or slide the training window sequentially.

Key Idea

Evaluates models for forecasting without leaking future information.

7. Time Series Cross-Validation (Rolling / Forward)

Description

- Used when observations are time-dependent.
- Ensures no “future” data leaks into training.

Formulation

For $t_0 < t_1 < \dots < t_n$:

$$S_{\text{Tr}}^{(k)} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{t_k}, y_{t_k})\}, \quad S_{\text{Te}}^{(k)} = \{(\mathbf{x}_{t_k+1}, y_{t_k+1})\}$$

Remarks

- Mimics real forecasting setting.

References

- 1 James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An introduction to statistical learning with applications in R. (2013).
- 2 Cross Validation in Machine Learning:
<https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/>

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY

PROF. ARINDAM BANERJEE

Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARGPUR



Module 05: Statistical Learning Theory-II

Lecture 04: Regression vs Classification

What is a Classification Problem?

Definition

A **classification problem** aims to assign each input $\mathbf{x}$ to one of a set of discrete classes $\mathcal{Y} = \{1, 2, \dots, K\}$.

- Examples:
 - Email $\rightarrow$ Spam or Not Spam
 - Image $\rightarrow$ Cat, Dog, or Car
 - Medical scan $\rightarrow$ Benign or Malignant
- The model learns a rule $h : \mathbf{x} \mapsto y$ from labeled data.
- Note that x may represent a vector.

Learning to Classify

- The learning algorithm estimates a function that separates classes in feature space.
- Training data: $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $y_i \in \mathcal{Y}$.
- The classifier predicts:

$$\hat{y} = \arg \max_{c \in \mathcal{Y}} P(y = c \mid \mathbf{x})$$

- This produces the most likely class given the input features.

Goal

Minimize the probability of misclassification on unseen data.

Why Predict Class Probabilities?

- Instead of only outputting a class label, a good classifier estimates:

$$P(y = c \mid \mathbf{x})$$

- These probabilities express the model's **confidence**.
- They allow flexible decision thresholds (e.g., alert if $P > 0.8$).
- Probabilistic outputs can be combined in ensembles or Bayesian reasoning.

Key Idea

Classification is best understood as estimating class probabilities and making informed decisions based on them.

Specific Example: Detecting Cars in Images

Goal

Given an image, we want to decide whether it **contains a car or not**.

- This is a decision between **discrete categories**: Car vs. Not Car.
- Hence, it is naturally framed as a **classification problem**.
- The classifier learns from labeled examples of images.

Why Classification Works Well

- A classifier learns a mapping:

$$h(\text{Image}) \rightarrow \{\text{Car}, \text{Not Car}\}$$

- The learning algorithm finds a decision boundary separating the classes.
- It can use features such as shapes, colors, and edges extracted from the image.

Result

The model generalizes to unseen images and predicts whether a car is present.

Why Predicting Probabilities is Better

- Instead of a hard decision, classifiers often output

$$P(\text{Car} \mid \text{Image})$$

— the probability that the image contains a car.

- This captures the model's **confidence** in its prediction.
- It allows flexible decision thresholds (e.g., alert only if $P > 0.9$).
- Probabilistic outputs are essential for combining models or risk-based decisions.

Key Idea

Classification with probabilistic outputs gives both the decision and the confidence level.

Regression: Key Ideas

- **Goal:** Predict a continuous-valued output.
- **Examples:** Predicting house prices, temperatures, stock values.
- **Model:** $y = f(x) + \epsilon$
- **Loss Function:** Mean Squared Error (MSE)

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Common Algorithms:** Linear Regression, Polynomial Regression.

Classification: Key Ideas

- **Goal:** Predict discrete class labels.
- **Examples:** Spam detection, image recognition, disease diagnosis.
- **Model:** $P(y|x)$ for discrete y .
- **Loss Function:** Cross-Entropy / Log Loss

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i})]$$

- **Common Algorithms:** Logistic Regression, Decision Trees, Neural Networks.

Differences Between Classification and Regression

Conceptual Difference

- **Regression:** Predicts a *continuous numerical value*.
- **Classification:** Predicts a *discrete class label*.

Mathematical Formulation

- **Regression:**

$$y_i = f(\mathbf{x}_i) + \epsilon_i, \quad y_i \in \mathbb{R}$$

- **Classification:**

$$P(y_i = c | \mathbf{x}_i) = g_c(\mathbf{x}_i; \theta), \quad y_i \in \{1, 2, \dots, K\}$$

Differences Between Classification and Regression

Learning Objective

- **Regression:** Minimize Mean Squared Error (MSE)

$$\min_{\theta} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{P}_{y_i})^2 \right)$$

- **Classification:** Minimize Cross-Entropy Loss

$$\min_{\theta} \left(-\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^K \mathbf{1}\{y_i = c\} \log P(y_i = c | \mathbf{x}_i) \right)$$

Note:

Note that Cross-Entropy Loss is the negative of the likelihood function $\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^K \mathbf{1}\{y_i = c\} \log P(y_i = c | \mathbf{x}_i)$

Motivation for Logistic Regression

Problem

Linear regression can output values outside $[0, 1]$, which are not valid probabilities.

Solution

Use a function that maps any real number to the labels $\{0, 1\}$:

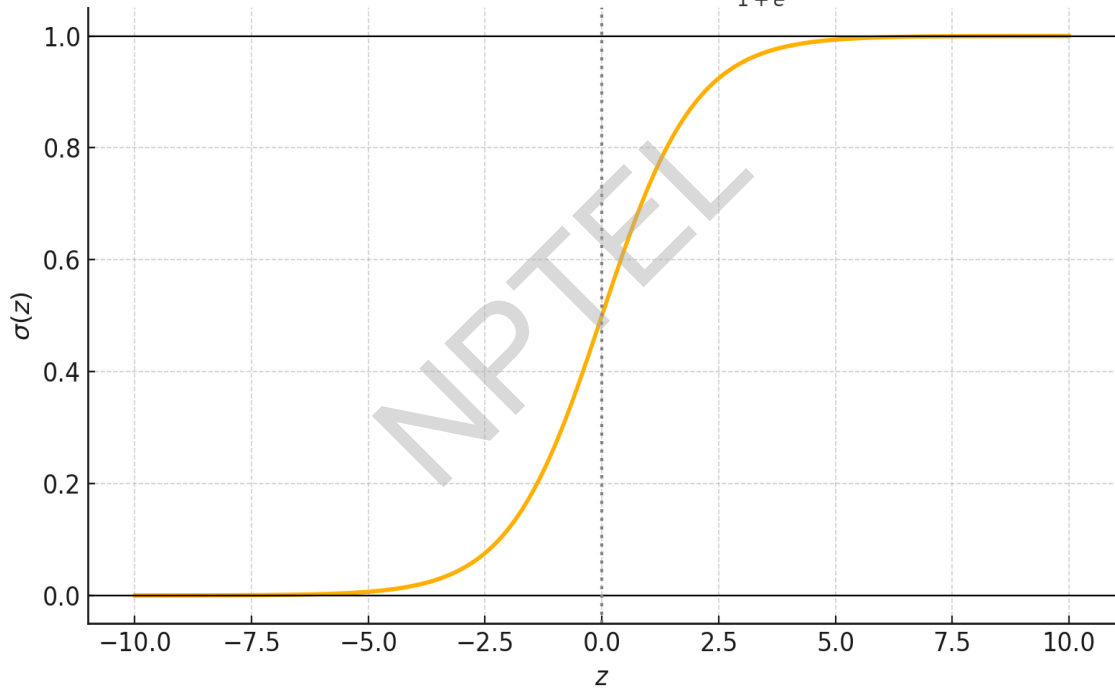
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$P(y = 1|x) = \sigma(w^T x + b)$ where x may represent a vector.

Interpretation

- $\sigma(z)$ gives the probability that $y = 1$ given x
- Decision boundary at $\sigma(z) = 0.5 \Rightarrow w^T x + b = 0$

Sigmoid Function $\sigma(z) = \frac{1}{1 + e^{-z}}$



Model and Rule

Model

For each i , we have $y_i \in \{0, 1\}$:

$$P(\hat{P}_{y_i}) = P(y_i = 1|x_i) = \sigma(w^T x_i + b)$$

$$P(y_i = 0|x_i) = 1 - \hat{P}_{y_i}$$

Rule Function

$$\hat{P}_{y_i} = h_w(x_i) = \frac{1}{1 + e^{-(w^T x_i + b)}}$$

Step 1: Constructing the Likelihood

For a Single Observation

$$P(y_i|x_i) = (\hat{P}_{y_i})^{y_i} (1 - \hat{P}_{y_i})^{1-y_i}$$

For All Observations (Assuming i.i.d.)

$$L(w, b) = \prod_{i=1}^n (\hat{P}_{y_i})^{y_i} (1 - \hat{P}_{y_i})^{1-y_i}$$

Log-Likelihood

$$\ell(w, b) = \sum_{i=1}^n [y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i})]$$

Step 2: Defining the Loss Function

Negative Log-Likelihood (to Minimize)

$$J(w, b) = -\ell(w, b)$$
$$J(w, b) = -\sum_{i=1}^n \left[y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i}) \right]$$

Simplified Using $\hat{P}_{y_i} = \sigma(w^T x_i + b)$

$$J(w, b) = -\sum_{i=1}^n \left[y_i \log \sigma(w^T x_i + b) + (1 - y_i) \log(1 - \sigma(w^T x_i + b)) \right]$$

Step 3: Minimizing the Loss Function

The Requirement for Numerical Methods

- 1 The derivative of the loss function cannot be expressed in a closed form.
- 2 Numerical Method (Gradient Descent) is used for minimizing the Loss Function.

Gradient Descent (Single Variable)

General Setting

We want to minimize a differentiable loss function:

$J(\theta) : \mathbb{R} \rightarrow \mathbb{R}$, where θ represents the model parameters.

Update Rule

The iterative parameter update is defined by:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{dJ(\theta^{(t)})}{d\theta}$$

- α is the learning rate (a small positive constant),
- $\frac{d^t J(\theta)}{d\theta}$ is derivative of $J(\theta)$ with respect to θ .

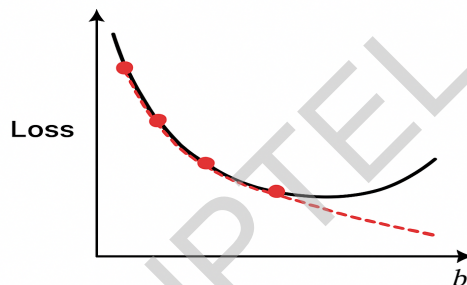
The algorithm stops when:

$$\left| \frac{dJ(\theta)}{d\theta} \right| < \varepsilon,$$

The gradient is close to zero and parameters are near a minimum.

Gradient Descent

Gradient descent in logistic regression



Gradient descent iteratively reduces the loss $J(b)$ — moving downhill.

For a differentiable single-variable function $J(\theta)$, a necessary condition for $\theta = \theta^*$ to be a local minimum is that $\left. \frac{dJ(\theta)}{d\theta} \right|_{\theta=\theta^*} = 0$. In the neighborhood of θ^* , the derivative satisfies $\frac{dJ(\theta)}{d\theta} < 0$ for $\theta < \theta^*$, and $\frac{dJ(\theta)}{d\theta} > 0$ for $\theta > \theta^*$, indicating that the function decreases to the left of the local minimum and increases to the right.

Gradient Descent — Mathematical Formulation

General Setting

We want to minimize a differentiable loss function:

$J(\theta) : \mathbb{R}^d \rightarrow \mathbb{R}$, where $\theta = (\theta_1, \theta_2, \dots, \theta_d)$ represents the model parameters.

Update Rule

The iterative parameter update is defined by:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} J(\theta^{(t)}),$$

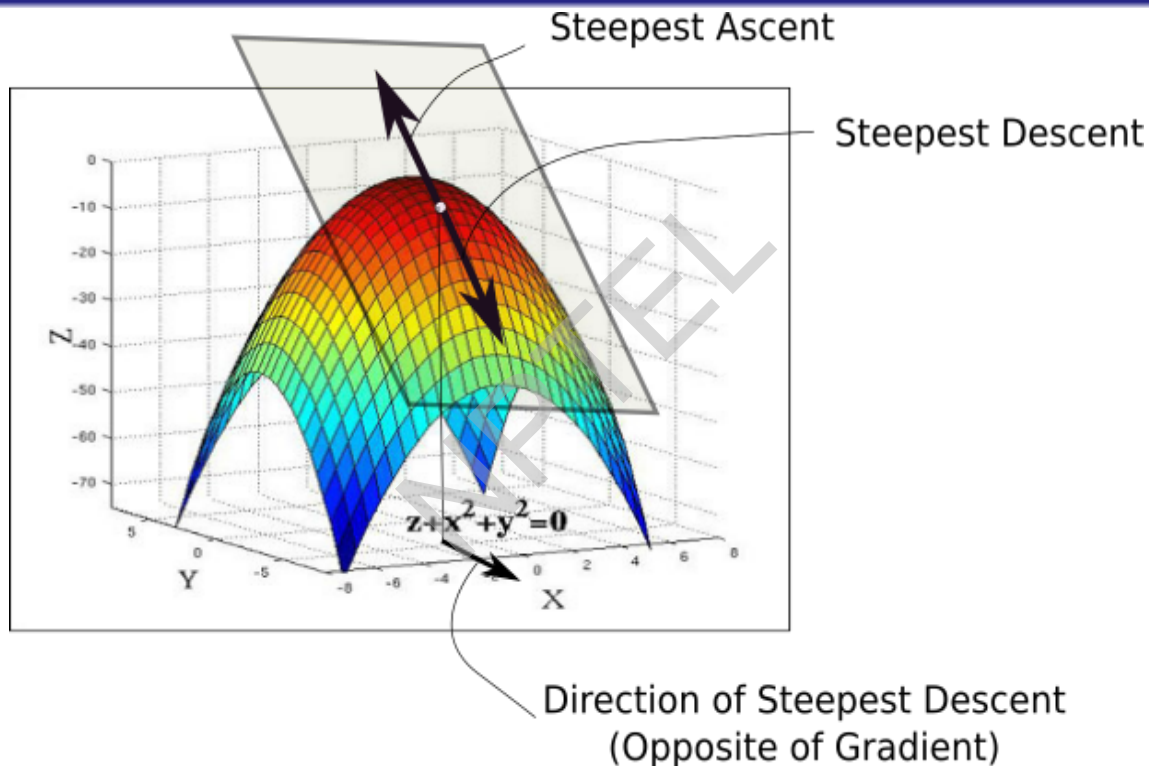
- α is the learning rate (a small positive constant),
- $\nabla_{\theta} J(\theta^{(t)})$ is the gradient vector of partial derivatives:

$$\nabla_{\theta} J(\theta) = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_d} \right]^T.$$

The algorithm stops when:

$$\|\nabla_{\theta} J(\theta)\| < \varepsilon,$$

The gradient is close to zero and parameters are near a minimum.



Step 4: Gradient Derivation

Derivative of Sigmoid

$$\frac{d\sigma(z)}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$

Gradient of negative of Log-Likelihood w.r.t. w

$$\frac{\partial(-\ell)}{\partial w} = - \sum_{i=1}^n (y_i - \hat{P}_{y_i}) x_i$$

Gradient w.r.t. b

$$\frac{\partial(-\ell)}{\partial b} = - \sum_{i=1}^n (y_i - \hat{P}_{y_i})$$

Note that : $l(w, b) = \sum_{i=1}^n \left[y_i \log \sigma(w^T x_i + b) + (1 - y_i) \log(1 - \sigma(w^T x_i + b)) \right]$

Step 5: Gradient Descent Intuition (Logistic Regression)

Key Idea

We want to find w and b that maximize the log-likelihood (or equivalently minimize the negative log-likelihood):

$$-\ell(w, b) = -\sum_{i=1}^n \left[y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i}) \right], \quad \hat{P}_{y_i} = \sigma(w^T x_i + b)$$

Gradient Directions

Using

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)),$$

we get:

$$\frac{\partial(-\ell)}{\partial w} = -\sum_{i=1}^n (y_i - \hat{P}_{y_i}) x_i, \quad \frac{\partial(-\ell)}{\partial b} = -\sum_{i=1}^n (y_i - \hat{P}_{y_i})$$

These terms tell us ****how the loss changes**** with w and b .

Step 6: Gradient Descent Update Rule

Gradient Descent Updates

Starting from initial guesses $w^{(0)}$, $b^{(0)}$, we move *against* the gradient:

$$w^{(t+1)} = w^{(t)} - \alpha \frac{\partial(-\ell)}{\partial w},$$
$$b^{(t+1)} = b^{(t)} - \alpha \frac{\partial(-\ell)}{\partial b},$$

where α is the learning rate.

Intuition

At each step:

- $(y_i - \hat{P}_{y_i})$ measures prediction error.
- x_i scales how much each feature contributes.
- The gradient “nudges” w and b toward smaller loss values.

Example (2D Case)

Given Data

$$x_1 = [0, 1], y_1 = 0 \quad \text{and} \quad x_2 = [1, 1], y_2 = 1$$

Initialize $w = [0, 0]^T$, $b = 0$, learning rate $\eta = 0.1$

Iteration 1

$$\hat{y}_1 = \sigma(0) = 0.5, \quad \hat{y}_2 = \sigma(0) = 0.5$$

Gradients:

$$\frac{\partial \ell}{\partial w} = (0 - 0.5)[0, 1] + (1 - 0.5)[1, 1] = [0.5, 0]$$

$$\frac{\partial \ell}{\partial b} = (0 - 0.5) + (1 - 0.5) = 0$$

Update:

$$w = [0, 0] + 0.1[0.5, 0] = [0.05, 0]$$

Interpretation

The model adjusts weights toward the correct class direction.

Equation

$$w^T x + b = 0$$

In 2D Example

If $w = [w_1, w_2]$, $b = \text{intercept}$:

$$w_1 x_1 + w_2 x_2 + b = 0 \Rightarrow x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

Meaning

Separates regions where $P(y = 1|x) > 0.5$ and $P(y = 0|x) < 0.5$.

Summary of Logistic Regression

Key Steps

- 1 Define Rule $h_w(x) = \sigma(w^T x + b)$
- 2 Construct likelihood $L(w, b)$
- 3 Take log-likelihood $\ell(w, b)$
- 4 Derive gradients and update using gradient ascent
- 5 Use boundary $w^T x + b = 0$ for classification

Output

Probabilistic, interpretable, linear decision boundary in feature space.

References

- 1 Kleinbaum, D.G. and Klein, M., 2002. Logistic regression: a self-learning text. New York, NY: Springer New York.
- 2 Intro to optimization in deep learning: Gradient Descent By Ayoosh Kathuria and Shaoni Mukherjee: <https://www.digitalocean.com/community/tutorials/intro-to-optimization-in-deep-learning-gradient-descent>
- 3 What is logistic regression? By Fangfang Lee: <https://www.ibm.com/think/topics/logistic-regression>

Thank you





SWAYAM NPTEL COURSE ON STATISTICAL FOUNDATION FOR BIG DATA ANALYSIS

BY

PROF. ARINDAM BANERJEE

Department of Mathematics
INDIAN INSTITUTE OF TECHNOLOGY KHARGPUR

Module 05: Statistical Learning Theory-II

Lecture 05: Inference on Logistic Regression



Logistic Regression Model

Binary Outcome

$$y_i \in \{0, 1\}$$

Class Probabilities

$$P(y_i = 1 \mid x_i) = \sigma(w^T x_i + b)$$

$$P(y_i = 0 \mid x_i) = 1 - \sigma(w^T x_i + b)$$

Sigmoid Function

The function σ

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Rule Function

$$h_w(x_i) = \frac{1}{1 + e^{-(w^T x_i + b)}}$$

Coefficients and Vector Form

Statistical vs. Model Coefficients

- Statistical notation uses coefficients: β_j
- Model notation uses parameters inside:

$$\sigma(w^T x_i + b)$$

Vector Form of Parameters

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

Likelihood and Loss Function

Likelihood (i.i.d. observations)

$$L(w, b) = \prod_{i=1}^n (\hat{P}_{y_i})^{y_i} (1 - \hat{P}_{y_i})^{1-y_i}$$

Log-Likelihood

$$\ell(w, b) = \log(L(w, b)) = \sum_{i=1}^n \left[y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i}) \right]$$

Negative Log-Likelihood (Loss)

$$J(w, b) = -\ell(w, b)$$

$$J(w, b) = - \sum_{i=1}^n \left[y_i \log \sigma(w^T x_i + b) + (1 - y_i) \log(1 - \sigma(w^T x_i + b)) \right]$$

Note

- In logistic regression, we obtain the values of b and w by maximizing $J(w, b)$ or equivalently minimizing $\ell(w, b)$.
- $\hat{P}_{y_i}$ denotes the model's predicted probability for the true class y_i .

For binary classification, if $\hat{p}_i = P(y = 1 \mid x_i)$, then $\hat{P}_{y_i} = \hat{p}_i$ when $y_i = 1$ and $\hat{P}_{y_i} = 1 - \hat{p}_i$ when $y_i = 0$.

Likelihood Ratio Test

Parameter Spaces

- Null hypothesis H_0 : The parameter θ lies in the subset Θ_0 of the parameter space Θ .
- Alternative hypothesis H_1 : The parameter θ lies in the complement of Θ_0 , i.e., $\theta \in \Theta \setminus \Theta_0$.

Likelihood Ratio Test

The likelihood ratio test compares the fit of two models:

- Reduced model (under H_0)
- Full model

Likelihood Function

Likelihood Function

Let $X = (X_1, X_2, \dots, X_n)$ represent independent observations from a probability distribution with the probability density function $f(x | \theta)$.

$$L(\theta; X) = \prod_{i=1}^n f(X_i | \theta)$$

Log-Likelihood Function

The log-likelihood function is:

$$\ell(\theta; X) = \log L(\theta; X) = \sum_{i=1}^n \log f(X_i | \theta)$$

Maximum Likelihood Estimators

The Maximum Likelihood Estimators are:

$$\hat{\theta}_0 = \arg \max_{\theta \in \Theta_0} \ell(\theta), \quad \hat{\theta}_1 = \arg \max_{\theta \in \Theta} \ell(\theta)$$

Likelihood Ratio and Test Statistic

Likelihood Ratio

The likelihood ratio is:

$$\Lambda(X) = \frac{\sup_{\theta \in \Theta_0} L(\theta; X)}{\sup_{\theta \in \Theta} L(\theta; X)} = \frac{L(\hat{\theta}_0; X)}{L(\hat{\theta}_1; X)}$$

Test Statistic

The likelihood ratio test statistic is:

$$D = -2 \log \Lambda(X) = 2(\ell(\hat{\theta}_1; X) - \ell(\hat{\theta}_0; X))$$

Asymptotic Distribution of Likelihood Ratio Test Statistic

Asymptotic Distribution

Under regularity conditions, if the null hypothesis H_0 is true:

$$D = -2 \log \Lambda(X) \xrightarrow{d} \chi_k^2 \text{ as the sample size } n \rightarrow \infty$$

where χ_k^2 is a chi-squared distribution with $k = \dim(\Theta) - \dim(\Theta_0)$ degrees of freedom.

Decision Rule

$$\text{Reject } H_0 \text{ if } D > \chi_{k, 1-\alpha}^2$$

where α is the significance level (typically 0.05).

Likelihood Ratio Test for Logistic Regression

Logistic Regression Model

The logistic regression model is given by: $\log \left(\frac{P(y=1)}{P(y=0)} \right) = b + w_1x_1 + w_2x_2 + \cdots + w_dx_d$ where $P(y = 1)$ is the probability of success, and $x_1, x_2, \dots, x_d$ are the predictors.

Hypothesis

$H_0 : w_1 = w_2 = \cdots = w_d = 0$ vs. $H_1 : \text{All coefficients } w_1, w_2, \dots, w_d \text{ are unrestricted}$

Full Model (under H_1)

$$\log \left(\frac{P(y = 1)}{P(y = 0)} \right) = b + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

Reduced Model (under H_0)

The reduced model excludes some predictors:

$$\log \left(\frac{P(y = 1)}{P(y = 0)} \right) = b + w_1x_1 + w_2x_2 + \cdots + w_tx_t \quad (\text{where } t < d)$$

Fitted Log-Likelihoods for Logistic Regression

Log-Likelihood for the Full Model (H_1)

$$\ell_1 = \log L(\hat{\theta}_1)$$

Log-Likelihood for the Reduced Model (H_0)

$$\ell_0 = \log L(\hat{\theta}_0)$$

Log-Likelihood Difference

$$D = 2(\ell_1 - \ell_0)$$

The test statistic D is twice the difference in the log-likelihoods.

Likelihood Ratio Test Statistic

Likelihood Ratio Statistic

The likelihood ratio statistic is:

$$D = -2 \log \Lambda = 2(\ell_1 - \ell_0)$$

Chi-Square Distribution

This statistic follows a chi-square distribution with $k = d - t$ degrees of freedom, where:

- d is the number of coefficients in the full model.
- t is the number of coefficients in the reduced model.

Degrees of Freedom

Degrees of Freedom

The degrees of freedom k for the chi-square distribution test statistic is the difference in the number of parameters between the full and reduced models:

$$k = \dim(\Theta_1) - \dim(\Theta_0)$$

For Logistic Regression

In logistic regression:

$$k = \text{number of parameters excluded in the reduced model}$$

The degrees of freedom correspond to the number of parameters in the full model that are excluded in the reduced model.

Decision Rule

Critical Value

The critical value from the chi-square distribution with k degrees of freedom is:

$$\chi_{k,1-\alpha}^2$$

Decision Rule

Reject H_0 if $D > \chi_{k,1-\alpha}^2$

If the test statistic D exceeds the critical value, reject the null hypothesis, implying that the additional predictors significantly improve the model.

Logistic Regression Models and Hypotheses

Setup

Consider a logistic regression model to predict the probability of disease ($y = 1$) based on two predictors:

- Age (x_1)
- Blood Pressure (x_2)

The models to be compared are:

- ****Full Model****: Includes both predictors x_1 (age) and x_2 (blood pressure)
- ****Reduced Model****: Includes only x_1 (age), with x_2 (blood pressure) excluded.

Hypotheses

Null Hypothesis H_0 : Blood pressure does not improve the model, i.e., the coefficient w_2 for blood pressure is zero:

$$H_0 : w_2 = 0$$

Alternative Hypothesis H_1 : Blood pressure does improve the model, i.e., the coefficient w_2 for blood pressure is non-zero:

$$H_1 : w_2 \neq 0$$

Dataset for Logistic Regression

Dataset Example

The dataset includes the following variables:

Age(x_1)	Blood Pressure(x_2)	Disease(y)
30	120	0
40	130	0
50	140	1
60	150	1
70	160	1

Log-Likelihoods from the Fitted Models

- Log-likelihood of the **full model**:

$$\ell_1 = -0.005017802718327788$$

- Log-likelihood of the **reduced model** (H_0):

$$\ell_0 = -3.3650583350362826$$

After fitting the logistic regression models in Python, the log-likelihoods are computed using the closed-form formula:

$$\ell(w, b) = \log(L(w, b)) = \sum_{i=1}^n \left[y_i \log \hat{P}_{y_i} + (1 - y_i) \log(1 - \hat{P}_{y_i}) \right]$$

Calculating the Likelihood Ratio Test Statistic

Likelihood Ratio Test Statistic

The test statistic is calculated as:

$$\begin{aligned} D &= 2(\ell_1 - \ell_0) \\ &= 2(-0.005017802718327788 - (-3.3650583350362826)) \\ &= 2(3.360040532317955) = 6.72008106463591 \end{aligned}$$

$$D \approx 6.720$$

Degrees of Freedom

Degrees of Freedom

The number of parameters in each model:

- Full model (H_1): 3 parameters (b, w_1, w_2)
- Reduced model (H_0): 2 parameters (b, w_1)

Thus:

$$k = 3 - 2 = 1$$

Decision Rule

Critical Value from Chi-Square Distribution

For $k = 1$ degree of freedom and significance level $\alpha = 0.05$:

$$\chi^2_{1,0.95} = 3.841$$

Decision Rule

Reject H_0 if $D > \chi^2_{1,0.95}$

Otherwise, fail to reject H_0 .

Final Decision

Conclusion

Since

$$D = 6.720 > 3.841,$$

we **reject** H_0 .

The full model **does** provide a significantly better fit than the reduced model at the 5% significance level.

Logistic Regression: Classification Rule

Decision Rule in Logistic Regression

In logistic regression, we aim to find a decision rule $h_S(x)$ that assigns probabilities to the response variable Y , based on the features $X = (X_1, X_2, \dots, X_n)$.

Threshold for Classification

The rule $h_S(x)$ outputs a probability, and we use a threshold, typically 0.5, to classify Y as:

$$Y = \begin{cases} 1 & \text{if } h_S(x) > 0.5 \\ 0 & \text{if } h_S(x) \leq 0.5 \end{cases}$$

Linear Regression: Model and Rule

Linear Regression Model

In linear regression, we model a dependent variable Y using the rule $h_S(x)$, where $h_S(x)$ is a linear function of the independent variables $X = (X_1, X_2, \dots, X_n)$.

Linear Rule

The rule $h_S(x)$ in linear regression takes the form:

$$Y = h_S(x) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Supervised vs. Unsupervised Learning

Supervised Learning

Both logistic regression and linear regression are examples of **supervised learning**, where the target variable Y is dependent on the features $X_1, X_2, \dots, X_n$.

- The goal is to learn from a rule $h_S(X)$ in terms of X to predict Y .
- The target variable Y is chosen to be the dependent variable, making it a supervised learning task.

Unsupervised Learning

In contrast, unsupervised learning arises when no clear distinction is made between the features, and the goal is to uncover patterns or structure from the data without a predefined dependent variable.

- We explore unsupervised learning in details in Module 8.

References

- 1 Kleinbaum, D.G. and Klein, M., 2002. Logistic regression: a self-learning text. New York, NY: Springer New York.
- 2 Intro to optimization in deep learning: Gradient Descent By Ayoosh Kathuria and Shaoni Mukherjee: <https://www.digitalocean.com/community/tutorials/intro-to-optimization-in-deep-learning-gradient-descent>
- 3 What is logistic regression? By Fangfang Lee: <https://www.ibm.com/think/topics/logistic-regression>
- 4 Wald test by Marco Taboga: <https://www.statlect.com/fundamentals-of-statistics/Wald-test>

Thank you

